

CMPSCI 282

Advanced Data Structures

Overall Goal of COMP182

- At the end of COMP182, you should have basic skills to approach a reasonably complex problem and ask design questions first:
 - What entities/objects would be appropriate to model this problem?
 - How would these objects interact to solve the problem?
 - What data structure would be the most appropriate or efficient to use?
- Fundamentally different problem-solving approach:
 - Bottom-up
- “Smart data structure and dumb code work a lot better than the other way around”.

Overall Goal of COMP182

- By the end of COMP282, you should be able to approach a reasonably complex problem and ask design questions first:
 - What entities/objects would be appropriate to model this problem?
 - How would these objects interact to solve the problem?
 - What data structure would be the most appropriate or efficient to use?
 - Main memory structures: hash tables, trees.
 - Architectural foundations for files.
 - Large-scale sorting.
 - Hash-based persistent structures.
 - Indexed files.
 - Introduction to databases.

Data Structures & Algorithms

- A data structure is an arrangement of data in a computer's memory (or sometimes on a disk)
- Algorithms are step by step instructions used to manipulate the data in given structure in various ways.

Introduction

- Rare to deal with only one instance
- Often need to deal with collections of objects
- Cannot always anticipate how many things we might need to deal with
- Languages usually provide support for arrays
- In Java:
 - `int n = 10;`
 - `int[] arr = new int[n];`

Problems With Arrays

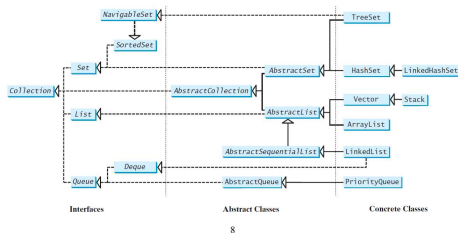
- Once created, capacity is fixed
- Cannot dynamically change its size (without rebuilding/copying)
- May not need entire capacity at certain points
- Must manually handle the organization
- Must have prior knowledge of the size before you create it

Better Solution: A List

- Better Solution: A List ADT (Abstract Data Type)
 - You don't have to be worry about lower-level details
 - Ex: when you use it, you just add the elements, you don't care how many elements are already there, this is the detail that we have to take care of it in class (abstract the details inside the class)
- Core Functionality (4 basic operations on any collection of data):
 - Adding elements
 - Retrieving elements
 - Removing elements
 - Automatic resizing

Java Collection Framework hierarchy, cont.

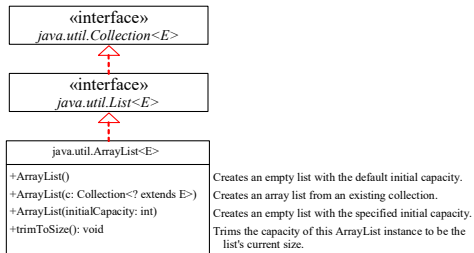
Set and List are subinterfaces of Collection.



ArrayList

- The `ArrayList` class is concrete implementation of the `List` interface.
- Supports random access through an index without inserting or removing elements from any place other than the end.
- A list can grow or shrink dynamically.
- An array is fixed once it is created. If your application does not require insertion or deletion of elements, the most efficient data structure is the array.

java.util.ArrayList



10

The ArrayList Class

You can create an array to store objects. But the array's size is fixed once the array is created. Java provides the ArrayList class that can be used to store an unlimited number of objects.

java.util.ArrayList<E>	
+ArrayList()	Creates an empty list.
+add(o: E) : void	Appends a new element o at the end of this list.
+add(index: int, o: E) : void	Adds a new element o at the specified index in this list.
+clear() : void	Removes all the elements from this list.
+contains(o: Object) : boolean	Returns true if this list contains the element o.
+get(index: int) : E	Returns the element from this list at the specified index.
+indexOf(o: Object) : int	Returns the index of the first matching element in this list.
+isEmpty() : boolean	Returns true if this list contains no elements.
+lastIndexOf(o: Object) : int	Returns the index of the last matching element in this list.
+remove(o: Object) : boolean	Removes the element o from this list.
+size() : int	Returns the number of elements in this list.
+remove(index: int) : boolean	Removes the element at the specified index x.
+set(index: int, o: E) : E	Sets the element at the specified index.

11

Generic Type

ArrayList is known as a generic class with a generic type E. You can specify a concrete type to replace E when creating an ArrayList. For example, the following statement creates an ArrayList and assigns its reference to variable cities. This ArrayList object can be used to store strings.

```
ArrayList<String> cities = new ArrayList<String>();
```

```
ArrayList<String> cities = new ArrayList<>();
```

12

Differences and Similarities between Arrays and ArrayList

Operation	Array	ArrayList
Creating an array/ArrayList	<code>String[] a = new String[10]</code>	<code>ArrayList<String> list = new ArrayList<>();</code>
Accessing an element	<code>a[index]</code>	<code>list.get(index);</code>
Updating an element	<code>a[index] = "London";</code>	<code>list.set(index, "London");</code>
Returning size	<code>a.length</code>	<code>list.size();</code>
Adding a new element		<code>list.add("London");</code>
Inserting a new element		<code>list.add(index, "London");</code>
Removing an element		<code>list.remove(index);</code>
Removing an element		<code>list.remove(Object);</code>
Removing all elements		<code>list.clear();</code>

13

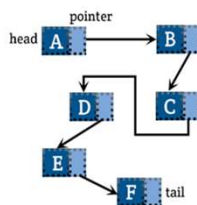
Linked List

- Options for implementing an ADT (List)
 - Array
 - Has a fixed size
 - Data must be shifted during insertions and deletions
 - Linked list
 - Is able to grow in size as needed
 - Does not require the shifting of items during insertions and deletions

Array

index	
0	A
1	B
2	C
3	D
4	E
5	F

Linked List

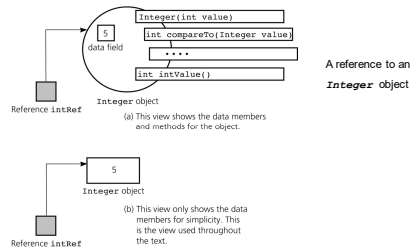


Object References

- A reference variable
 - Contains the location of an object
- Example


```
Integer intRef;
intRef = new Integer(5);
```
- As a data field of a class
 - Has the default value null
- A local reference variable to a method
 - Does not have a default value

Object References

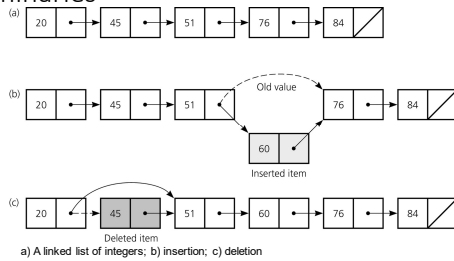


Object References

- When one reference variable is assigned to another reference variable, both references then refer to the same object

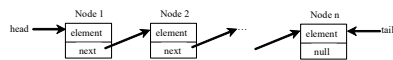

```
Integer p, q;
p = new Integer(6);
q = p;
```
- A reference variable that no longer references any object is marked for garbage collection

Preliminaries



Nodes in Linked Lists

A linked list consists of nodes. Each node contains an element, and each node is linked to its next neighbor. Thus a node can be defined as a class, as follows:



```
class Node<E> {
    E element;
    Node<E> next;

    public Node(E o) {
        element = o;
    }
}
```

20

Adding Three Nodes

The variable head refers to the first node in the list, and the variable tail refers to the last node in the list. If the list is empty, both are null. For example, you can create three nodes to store three strings in a list, as follows:

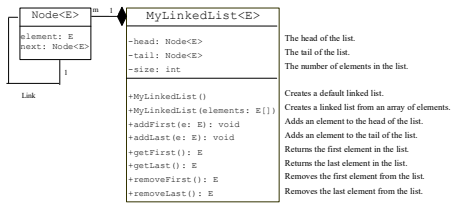
Step 1: Declare head and tail:

```
Node<String> head = null;
Node<String> tail = null;
```

The list is empty now

21

MyLinkedList



22

Traversing All Elements in the List

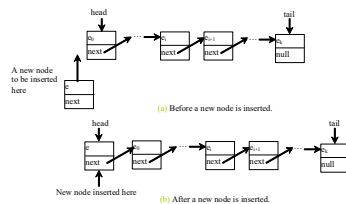
Each node contains the element and a data field named *next* that points to the next node. If the node is the last in the list, its pointer data field *next* contains the value *null*. You can use this property to detect the last node. For example, you may write the following loop to traverse all the nodes in the list.

```

Node<E> current = head;
while (current != null) {
    System.out.println(current.element);
    current = current.next;
}
  
```

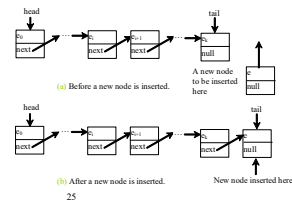
23

Implementing addFirst(E e)



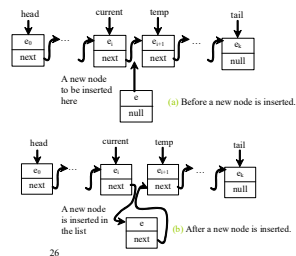
24

Implementing addLast(E e)



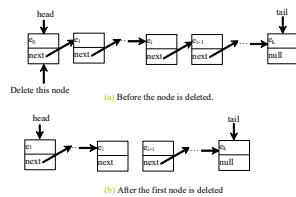
25

Implementing add(int index, E e)



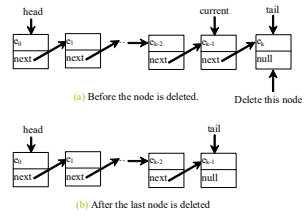
26

Implementing removeFirst()



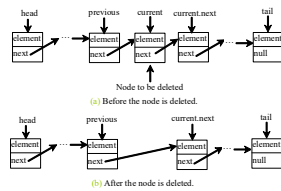
27

Implementing removeLast()



28

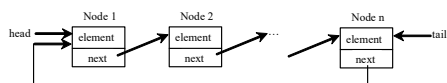
Implementing remove(int index)



29

Circular Linked Lists

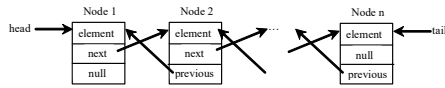
A *circular, singly linked list* is like a singly linked list, except that the pointer of the last node points back to the first node.



30

Doubly Linked Lists

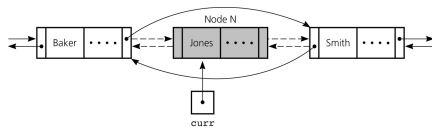
A *doubly linked list* contains the nodes with two pointers. One points to the next node and the other points to the previous node. These two pointers are conveniently called a *forward pointer* and a *backward pointer*. So, a doubly linked list can be traversed forward and backward.



31

Doubly Linked List

- To delete the node that `curr` references



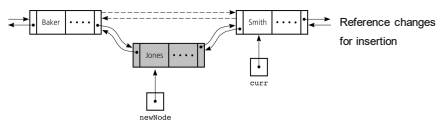
Reference changes for deletion

© 2011 Pearson Addison-Wesley. All rights reserved.

5.8-32

Doubly Linked List

- To insert a new node that `newNode` references before the node referenced by `curr`



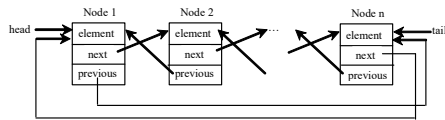
Reference changes for insertion

© 2011 Pearson Addison-Wesley. All rights reserved.

5.8-33

Circular Doubly Linked Lists

A *circular, doubly linked list* is doubly linked list, except that the forward pointer of the last node points to the first node and the backward pointer of the first pointer points to the last node.



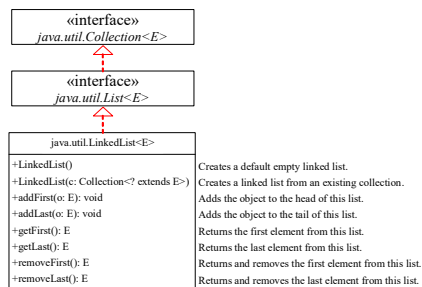
34

JCF - LinkedList

- LinkedList class is concrete implementation of the List interface.
- If your application requires the insertion or deletion of elements from any place in the list, you should choose LinkedList.

35

java.util.LinkedList



36

Time Complexity for ArrayList and LinkedList

Methods	MyArrayList/ArrayList	MyLinkedList/LinkedList
add(e: E)	$O(1)$	$O(1)$
add(index: int, e: E)	$O(n)$	$O(n)$
clear()	$O(1)$	$O(1)$
contains(e: E)	$O(n)$	$O(n)$
get(index: int)	$O(1)$	$O(n)$
indexOf(e: E)	$O(n)$	$O(n)$
isEmpty()	$O(1)$	$O(1)$
lastIndexOf(e: E)	$O(n)$	$O(n)$
remove(e: E)	$O(n)$	$O(n)$
size()	$O(1)$	$O(1)$
remove(index: int)	$O(n)$	$O(n)$
set(index: int, e: E)	$O(n)$	$O(n)$
addFirst(e: E)	$O(n)$	$O(1)$
removeFirst()	$O(n)$	$O(1)$

37

Stacks

Introduction

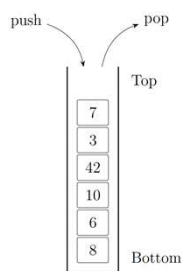
- Collection data structures such as lists are typically unstructured.
- A list, whether array based, a linked list, simply hold elements in an ordered manner.
- That is, there is a notion of a first element, second element, etc.
- However, that ordering is not necessarily structured, it is simply the order in which the elements were added to the collection.
- In contrast, you can impose a structured ordering by sorting a list
- Sorting a list, however, does not give the data structure itself any more structure.
- The interface would still allow a user to insert or rearrange the elements so that they are no longer sorted.

Stacks

- A stack is a data structure that stores elements in a last-in, first-out (or Last-In First-Out (LIFO)) manner.
 - That is, the last element to be inserted into a stack is the first element that will come out of the stack.
- The last dish that we added to the stack will be the first dish that we take of the stack.



Visualization



Stack

- ADT stack operations
 - Create an empty stack
 - Determine whether a stack is empty
 - Add a new item to the stack
 - Remove from the stack the item that was added most recently
 - Remove all the items from the stack
 - Retrieve from the stack the item that was added most recently

In Practice

- Don't roll your own!
- You generally do not want to implement your own stack, queue, etc.
 - JCF contains an implementation of a stack class called `Stack` (generic)
 - Derived from `Vector`
 - Includes methods: `peek`, `pop`, `push`, and `search`
 - `search` returns the 1-based position of an object on the stack

Simple Applications of the ADT Stack: Checking for Balanced Braces

- A stack can be used to verify whether a program contains balanced braces
 - An example of balanced braces
`abc{defg{ijk}{l{mn}}op}qr`
 - An example of unbalanced braces
`abc{def}){ghij{kl}m`

© 2011 Pearson Addison-Wesley. All rights reserved.

7A-44

Application: Algebraic Expressions

- When the ADT stack is used to solve a problem, the use of the ADT's operations should not depend on its implementation
 - Convert the infix expression to postfix form
 - Evaluate the postfix expression

© 2011 Pearson Addison-Wesley. All rights reserved.

7B-45

Evaluating Postfix Expressions

- A postfix calculator
 - Requires you to enter postfix expressions
 - Example: 2, 3, 4, +, *
 - When an operand is entered, the calculator
 - Pushes it onto a stack
 - When an operator is entered, the calculator
 - Applies it to the top two operands of the stack
 - Pops the operands from the stack
 - Pushes the result of the operation on the stack

© 2011 Pearson Addison-Wesley. All rights reserved.

7 B-46

Converting Infix Expressions to Equivalent Postfix Expressions

- An infix expression can be evaluated by first being converted into an equivalent postfix expression
- Facts about converting from infix to postfix
 - Operands always stay in the same order with respect to one another
 - An operator will move only "to the right" with respect to the operands
 - All parentheses are removed

© 2011 Pearson Addison-Wesley. All rights reserved.

7 B-47

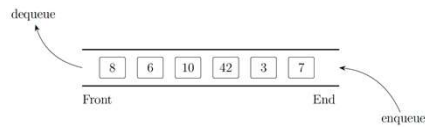
Queues

Queues

- A similar data structure is a queue which provides a First-In First-Out (FIFO) ordering of elements.
- As its name suggests, a queue can be thought of as a line.
- Elements enter the queue at one end (the “end” of the line) and are removed from the other end (the “front” of the line). Thus, the first element to enter a queue is the first element to be removed from the queue.

Core Functionality

- The two core operations in a queue are:
 - *enqueue* which adds an element to the queue at its end
 - *dequeue* which removes the element at the front of the queue



The Java Collections Framework Interface **Queue**

- JCF has a queue interface called **Queue**
- Derived from interface **Collection**
- Adds methods:
 - *element*: retrieves, but does not remove head
 - *offer*: inserts element into queue
 - *peek*: retrieves, but does not remove head
 - *poll*: retrieves and removes head
 - *remove*: retrieves and removes head

In Practice

- Don't roll your own!
- You generally do not want to implement your own stack, queue, etc.
- Java:
 - Deque <T> ("deck"): allows both insert and remove operations from both ends ("double ended queue")
 - It can either be used as a queue(first-in-first-out/FIFO) or as a stack(last-in-first-out/LIFO).
